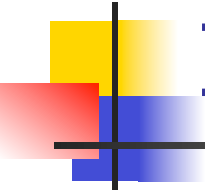


Corso di Crittografia

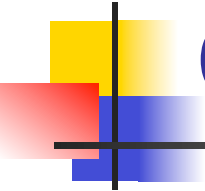
Prof. Dario Catalano

Message Authentication



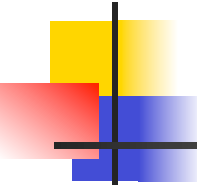
Introduzione

- Fino ad ora abbiamo parlato di sicurezza in termini di privacy.
- Garantire autenticita' e' un obiettivo forse ancora piu' importante. *ULTIMI ANNI*
- Non sempre la privacy e' una necessita'
- Quasi sempre e' necessario essere certi circa l'identita' del mittente.
- Oggi studieremo metodi (simmetrici) per realizzare tale obiettivo.



Obiettivi

- Il mittente S vuole inviare un msg M al destinatario R , in modo che se M e' corrotto, R possa rendersene conto.
- Viene garantita l'autenticita' del mittente.
- Viene protetta l'integrita' del messaggio.



Il nostro contesto

SIMMETRICA

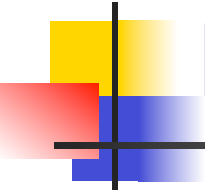
- S ed R hanno una chiave segreta k in comune.
- L'autentica e' per il "bene" di R, ma e' prodotta da S (utilizzando k e un algoritmo appropriato).
- L'autentica di un messaggio M produce un messaggio M' . *→ che sarà il "qualcosa"*
- Se R riceve M'' , dovrà essere in grado di:
 - Ricavare il messaggio originale M
 - Convincersi che M'' non e' autentico.

GUARDIAMO SOLO L'AUTENTICITÀ

MAC

Message Authentication Codes

- Spesso M' e' proprio il messaggio originale M , con aggiunta una stringa (di lunghezza fissa) *tag*. → STRINGA PER AUTENTICAZIONE
- Il tag serve a R per verificare l'autenticita' di M .
CON LE FIRME CLASSICHE NOI MODIFICIAMO IL DOCUMENTO. IL MAC AGGIUNGE QUALCOSA ESTERNO.
- In tali casi si parla di Message Authentication Codes (MAC).
- Se il messaggio non risulta autentico il comportamento di R dipende dal contesto.
SARÀ IL RICEVENTE A DECIDERE COSA FARE. NOI NOI LO CONSIDERIAMO PERCHÉ DIPENDE DAL CONTESTO



L'avversario

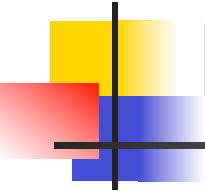
- L'avversario deve partecipare ad una comunicazione. → CANALE NON AFFIDABILE
 - Per sperare di avere informazioni, qualcosa deve essere inviata a R. TUTTA
- Supporremo che A controlla la rete di comunicazione.
- Obiettivo: capire come (e se) partecipare in modo attivo alla conversazione, aiuta l'avversario.

Autenticita' via Privacy

- Cifrare puo' garantire autenticita'?
- Se S vuole mandare M a R, S calcola $M' = \text{Enc}(M)$ e invia M' a R.
- R decifra, se il risultato ha senso allora esso viene dichiarato autentico.
- Ragionamento intuitivo: A non conosce M → non e' in grado di modificarlo in modo corretto.

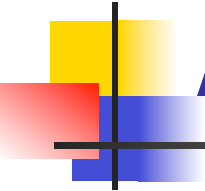
1. Che una dicitura ha senso?
2. Posso modificare il ciphertext?

3. Utilizzare mezzi complessi
ha senso? → NO



Eppure...

- Potremmo essere in grado di modificare il messaggio senza necessariamente conoscerlo (parzialmente o interamente).
- Che succede se utilizzassimo cifrari piu' potenti (o semplicemente diversi)?
 - Es. cifrari sicuri contro attacchi attivi.



Autenticita' vs Privacy

- Il problema non sta' nel cifrario utilizzato.
- Piu' semplicemente **gli obiettivi sono diversi**.
- Utilizzare per un dato problema, metodi progettati per risolverne altri e' sbagliato.
 - Sappiamo che obiettivi vogliamo realizzare, non come (o da che contesto) nascono tali obiettivi.
 - Le nostre definizioni sono studiate per obiettivi precisi e specifici.

ha senso categorizzare le cose su misura!

USARE GU STATI CREA UNA VARIABLE SECURITY SENSITIVE

LO STATO SAREBBE USATO

SOLO DAL PRIMO O

SAREBBE NECESSARIA UNA

SINCRONIZZAZIONE

MAC - Definizione

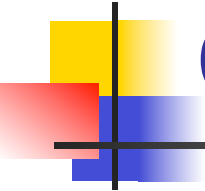
(KeyGen, MAC, VF)

- L'algoritmo KeyGen restituisce una stringa casuale (nell'insieme opportuno)
- MAC. Prende in input una chiave k , un messaggio $M \in \{0, 1\}^*$. Restituisce $tag \in \{0, 1\}^* \cup \{\perp\}$.
- ^{VERIFIER} VF (deterministico). Prende in input una chiave k , un messaggio $M \in \{0, 1\}^*$ e un $tag \in \{0, 1\}^*$. Restituisce un bit d

ATTACK

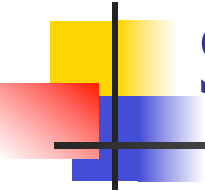
REPLAY CON MAC STATI
PERCARE LO STATO CAMBIA

I NON SI POSSONO FARE



Commenti

- Fino ad ora non abbiamo parlato di sicurezza.
- La sintassi descritta sembra imporre verificatori (destinatari) deterministici.
- In realta' avere destinatari a stati non e' impossibile ma problematico.
 - L'avversario controlla la comunicazione → non e' banale prevedere lo stato del destinatario.
- Eppure algoritmi di verifica a stati sono importanti per far fronte a certi tipi di attacco (es. replay attacks)



Sistemi Deterministici

- Per il caso dei cifrari abbiamo visto che I metodi deterministici non possono essere sicuri.
- Che possiamo dire per l'autentica dei messaggi?
- Vale la stessa restrizione?

I sistemi deterministici non sono un problema
perché non vogliamo indistinguibili

Sistemi Deterministici (cont.)

- Se MAC e' deterministico (e senza stati)
l'algoritmo di verifica e' sempre lo stesso

VF_k(M, tag)

tag' ← MAC_k(M) ←

lo RICALCOLO

if tag' = tag ∧ tag ≠ ⊥ return 1 else return 0.

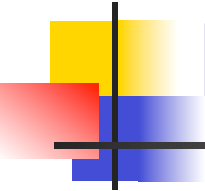
- Dunque per tali MAC non abbiamo bisogno di specificare un algoritmo di verifica.

06/11/2025

Verso una definizione di sicurezza

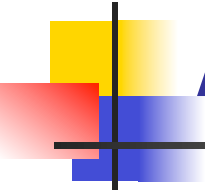
- Obiettivo: rivelare ogni tentativo di modifica da parte di A.
- Il destinatario dovrebbe considerare autentico solo un msg M che non e' stato alterato.
- Se A produce (M, tag) tali che $VF_k(M, \text{tag}) = 1$, diciamo che A produce un *falso*.
- Come possiamo formalizzare questo requisito?

IL FALSO NON DEVE AVERE NECESSARIAMENTE SENSO. → INOLTRE È PIÙ GENERALE



Requisiti

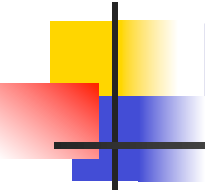
- Determinate proprietà sono certamente necessarie (es. segretezza della chiave)
- L'obiettivo però rimane la non falsificabilità'.
- Determinati "falsi" potrebbero non avere alcun significato.
- E' giusto considerarli veri e propri falsi?



Attacchi

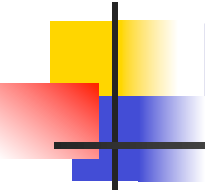
- Una volta stabiliti gli obiettivi restano da definire i tipi di attacchi che fronteggeremo.
- Dobbiamo stabilire a cosa ha accesso l'avversario prima di (tentare di) produrre un falso. → COPIE (MESSAGGI TAG) Poi Fa IL FALSO
- Possiamo definire diverse classi di avversari via via piu' potenti.

L'AVVERSAIO PUÓ ANCHE SCEGLIERE I MESSAGGI
DA CUI VUOLA FIDERE (M, TAG)



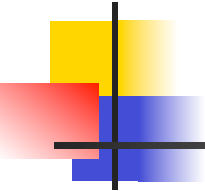
No message attack

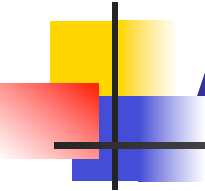
- A non ha la possibilità' di accedere a nulla di piu' che la descrizione degli algoritmi utilizzati.
- A deve produrre una coppia valida (M, tag) .
- Si tratta di un attacco non molto realistico.



Replay Attack

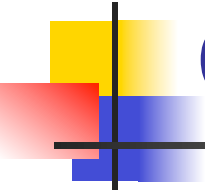
- A intercetta una coppia (valida) (M, tag) e la invia una seconda volta a R.
- Se il MAC e' deterministico e senza stati tale attacco e' inevitabile: cio' che R accetta la prima volta, verra' accettato ogni altra volta.
- Dobbiamo considerare questo un attacco importante?

- 
-
- In realta' non vi e' motivo perche' A debba vedere una sola coppia (M, tag)
 - Potrebbe avere a disposizione q_s coppie di tale tipo.
 - Tale numero puo' essere limitato in contesti specifici.
 - In generale, tratteremo q_s come una importante risorsa di A.



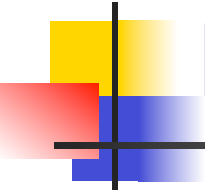
Attacchi a msg scelto (CMA)

- A potrebbe vedere q_s coppie sulle quali non ha alcun controllo.
- Alternativamente, potremmo immaginare di fornire ad A un oracolo che restituisce il tag opportuno ogni qualvolta A chiede un msg M.
- In tal caso parliamo di attacco a messaggio scelto (CMA).
- L'obiettivo diventa di produrre un falso MAC per un messaggio non richiesto all'oracolo.



Completiamo il modello

- A potrebbe essere in grado di produrre diversi candidati “falsi”.
- A vince se anche solo uno di tali candidati e' valido.
- Possiamo formalizzare tale capacita' permettendo ad A di accedere ad un oracolo di verifica.



Ricapitoliamo

- A può chiedere di autenticare un certo numero di messaggi.
 - A ha a disposizione q_s richieste di autentica.
- A può chiedere se una coppia del tipo (M, tag) è valida
 - A ha a disposizione q_v richieste di verifica.
- A vince se effettua una richiesta di verifica, su un messaggio per il quale non aveva ricevuto autentica, e l'algoritmo di verifica restituisce 1.



CHOSEN MESSAGE ATTACK

Definizione

UF: UNFORGIABILITY

MESSAGE AUTHENTICATION

MA: (KeyGen, MAC, VF) ← VERIFICO MAC

CREO MC

COPIA
M, v.

$\text{Esp}_{\text{MA}}^{\text{uf-cma}}(A)$

$k \leftarrow_R \text{KeyGen}();$ Esegui $A^{\text{MAC}_k(\cdot), \text{VF}_k(\cdot, \cdot)}$

if A fa una richiesta di verifica (M, tag) tale che

- L'oracolo di verifica risponde 1 SE È AUTENTICO

- A non aveva richiesto M all'oracolo MAC

Return 1

IMBROGLIA

else Return 0

2 ORACOLI
MAC
VERIFICA

$$\text{Adv}^{\text{uf-cma}}(A) = \Pr[\text{Esp}_{\text{MA}}^{\text{uf-cma}}(A) = 1]$$

$q \rightarrow \text{QUERY}$

$\$A \rightarrow$
SIGN

VERIFICA

Commenti

- E' importante separare q_s e q_v dal tempo di calcolo.
- In pratica le autentiche possono essere ottenute solo dal legittimo mittente.
- q_s e q_v possono essere limitate dal sistema.
- Adesso facciamo pratica con la definizione guardando un esempio.

$$M = \{0,1\}^{t \cdot \ell}$$

Esempio

- $F: \{0,1\}^k \times \{0,1\}^\ell \rightarrow \{0,1\}^L$ PRF

MAC_k(M)

if ($|M| \bmod \ell \neq 0 \vee |M| = 0$) ↑ return $\perp$ lunghezza max

Sia $M = M[1] \cdots M[n]$ // $|M[i]| = \ell$

for $i = 1, \dots, n$ $y_i \leftarrow F_k(M[i])$

$Tag \leftarrow y_1 \oplus \cdots \oplus y_n$

return Tag

FALLISCE SE TUTTI I BLOCCHI SONO UGUALI

- E' questo schema sicuro? No!

$$n = x || x || x$$

$$F_K(x) \oplus F_K(x) \oplus F_K(x)$$

$A(n) \rightarrow$ ^{over} ~~MAC~~ _{MAC} ^{RESSOURCE}

$$x \leftarrow \{0,1\}^L$$

$$n \leftarrow x || x$$

$$\text{tag} \leftarrow 0^L$$

$$\text{OVER}(n, \text{tag})$$

L'ATTACKANT FINISSE QU'IL

$$\text{Ver}_K(n, \text{tag})$$

$$\text{tag}' \leftarrow \text{MAC}_K(n)$$

$$\text{if } (\text{tag} == \text{tag}') \wedge (\text{tag} \neq \perp)$$

Output 1
else output 0

$$\text{Adv}(A) = \Pr[\text{Exp}^{\text{uf-cma}}(A) = 1] = 1$$

$$\underbrace{x || x}_n, \underbrace{0^L}_{\text{tag}}$$

$$\text{tag}' \leftarrow \underbrace{F_K(x) \oplus F_K(x)}_{0^L}$$

$$\times F: \{0,1\}^k \times \{0,1\}^L \rightarrow \{0,1\}^L \quad \text{PRF}$$

$$\text{MAC}_K(m)$$

$$m \in \mathcal{M} = \{0,1\}^{2L}$$

if $m \notin \mathcal{M}$ output $\perp$

$$\text{tag} \leftarrow F_K(m_0) || F_K(m_1) \quad m = m_0 || m_1, \quad m_0, m_1 \in \{0,1\}^L$$

Output tag

$A(nA)$

$$X, Y \leftarrow \{0, 1\}^{\ell} \quad X \neq Y; \quad r_1 \leftarrow X \parallel Y$$

$$\text{tag}_1 \leftarrow O^{\text{MAC}}(r_1)$$

$$\text{tag}_2 = \alpha \parallel \beta$$

$$r_2 \leftarrow Y \parallel X; \quad \text{tag}_2 \leftarrow \beta \parallel \alpha$$

$$O^{\text{VER}}(r_2, \text{tag}_2)$$

$$\text{Adv}(A) = \Pr[\text{Exp}^{\text{uf-cma}}(A) = 1] = 1$$

(3) $F: \{0, 1\}^k \times \{0, 1\}^{\ell+1} \rightarrow \{0, 1\}^{\ell} \quad \mu = \{0, 1\}^{2\ell}$

$\text{MAC}_F(\mu)$

$$\mu = r_1 \parallel r_2 \quad // \quad r_1, r_2 \in \{0, 1\}^{\ell}$$

$$\text{tag} \leftarrow F_k(r_1 \parallel 0) \parallel F_k(r_2 \parallel 1)$$

output tag

$$\text{tag}_1 = \alpha \parallel \beta$$

$$\text{tag}_2 = \gamma \parallel \delta$$

$$\text{tag}_3 = \alpha \parallel \delta$$

$A(nA)$

$$X, Y, Z, W \in \{0, 1\}^{\ell} \quad X \neq Y \neq Z \neq W$$

$$r_1 \leftarrow X \parallel Y; \quad r_2 \leftarrow Z \parallel W$$

$$\text{tag}_1 \leftarrow O^{\text{MAC}}(r_1); \quad \text{tag}_2 \leftarrow O^{\text{MAC}}(r_2)$$

$$r_3 \leftarrow X \parallel W \quad \text{tag}_3 \leftarrow [\text{tag}_1]_1 \parallel [\text{tag}_2]_2$$

$$O^{\text{VER}}(r_3, \text{tag}_3)$$

$$M_X \rightarrow F_K(X|0) \parallel F_K(Y|1)$$

$$\mu_2 \rightarrow F_K(z|0) \| F_K(w|1)$$

$$r_3 = x \parallel w \rightarrow F_k(x \parallel 0) \parallel F_k(w \parallel 1)$$

$$HAc_N \quad (M)$$

if $Y \bmod C \neq 0$ output 1

Sia $M = M_1 \parallel M_2 \parallel \dots \parallel M_m$

$$F_{02} \quad i = 1 \dots m \quad y_i \leftarrow F_{\psi}(m_i)$$
$$\text{tag} \leftarrow y_1 \parallel y_2 \parallel \dots \parallel y_m$$
 $A(\mu_{AC})$

AC)

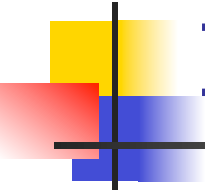
$m_i \leftarrow 0 \parallel 1^L$

Ex. $\rightarrow$ MAC (n)

$$\text{tag}_1 = \text{tag}_n \parallel \text{tag}_2$$
$$m_2 \sim 1 \parallel 0^c; \quad \text{tag}_2 \sim \text{tag}_{12} \parallel \text{tag}_{11}$$

over (M_2, Eug_2)

[illegible]



Il paradigma PRF-come-MAC

- PRF consentono di costruire MAC sicuri
- In realta' tale approccio e' ottimo.
- La sicurezza del MAC costruito nel modo che vedremo e' praticamente la stessa della PRF utilizzata (la dimostrazione non degrada di molto la sicurezza)



MAC da PRF

- $F: K \times D \rightarrow \{0,1\}^\tau$ fam. di funz.

KeyGen $k \leftarrow_R K$ return k	$\text{MAC}_k(M)$ If $(M \notin D)$ return $\perp$ Tag $\leftarrow F_k(M)$ return Tag
---	---

- Non specifichiamo l'algoritmo di verifica poiche' il metodo e' deterministico.



Teorema

- $F: K \times D \rightarrow \{0,1\}^\tau$ fam. di funz.
- $MA=(\text{KeyGen}, \text{MAC})$ il metodo appena descritto.
- A avversario che attacca MA facendo $(q_s + q_v)$ domande.
- Esiste un avversario B - contro F - che fa $(q_s + q_v)$ domande e tale che

$$Adv_{MA}^{uf-cma}(A) \leq Adv_F^{prf}(B) + \frac{q_v}{2^\tau}$$

PRF

TRANSITO: SE ROMPO LA PRF, ROMPO A TAG

$$F: \{0,1\}^k \times \{0,1\}^l \rightarrow \{0,1\}^{10}$$

mc 

tag

$$\frac{1}{2^{10}}$$

A avversario MAC

B avversario PRF

$B(F)$ ~~boole~~ 0, FLAG

$d \leftarrow 0$; $S \leftarrow \emptyset$

Esqui $A(n)$

if A chiede n all'oracolo MAC

B insira l'oracolo prf su input n
e insira l'output ($f(n)$) ad A

$S \leftarrow S \cup \{n\}$

if A chiede (n, tag) all'oracolo VER

if $(f(n) == \text{tag})$

rispondi 1 ad A

if $n \notin S$ $d \leftarrow 1$

else

rispondi 0 ad A

output d

$$\text{Adv}(B) = \Pr[\text{Esp}^{\text{P}^f, 1}(B) = 1] - \Pr[\text{Esp}^{\text{P}^f, 0}(B) = 1]$$

$$\Pr[\text{Esp}^{\text{P}^f, 1}(B) = 1] = \text{Adv}(A) \quad \text{domande di } A$$

$$\Pr[\text{Esp}^{\text{P}^f, 0}(B) = 1] \leq \frac{q_v}{2^c} \quad \leftarrow$$

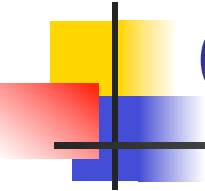
$d \rightarrow 1$ se A ha successo

$$\Pr[\text{Succ}] = \Pr[T_1 \vee T_2 \vee \dots \vee T_{q_v}] \leq$$

$$\leq \sum_i \Pr[T_i] = \frac{q_v}{2^c}$$

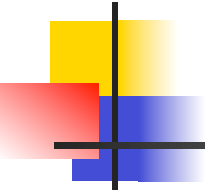
SPARA
CASO A

$$\Pr[T_i] = \frac{1}{2^c} \quad \forall i$$



CBC-MAC

- Si tratta di un metodo molto diffuso per costruire MAC a partire da un cifrario a blocchi $E: \{0,1\}^k \times \{0,1\}^n \rightarrow \{0,1\}^n$
- Metodo deterministico e senza stati.
- KeyGen si limita a restituire una stringa casuale di k bit.



$$\mathcal{M} = \{0,1\}^{tn}$$

MAC_k(M)

if $M \notin \mathcal{M}$ **return** $\perp$

// $\mathcal{M}$ spazio dei messaggi

Sia $M = M[1] \cdots M[m]$

// $|M[i]| = n$

~~$C[0] \leftarrow 0^n$~~ $C[0] \leftarrow 0^m$

for $i = 1, \dots, m$

$C[i] \leftarrow E_k(C[i-1] \oplus M[i])$

return $C[m]$ // Tag = $C[m]$

→ SOLO L'ULTIMO ELEMENTO

$$|M| = 150$$

0000111

→ tog.

$$M_1 = 000111$$

~~Devo~~ scegliere m in anticipo
se ho m grandi iterazioni ho tante del cib.

$m = 128$

$m = 4$

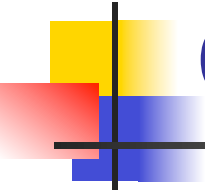
Teorema

- $E: \{0,1\}^k \times \{0,1\}^n \rightarrow \{0,1\}^n$ cifrario a blocchi
- $\{0,1\}^{nm}$ spazio dei messaggi
- A avversario contro CBC-MAC che fa q richieste di autentica e 1 richiesta di verifica.
- Esiste B che attacca E come PRP, che fa $q+1$ domande e tale che

PERMUTAZIONE CASUALE

$$\text{Adv}_{MAC}^{uf-cma}(A) \leq \text{Adv}_E^{prp-cpa}(B) + \frac{m^2 q^2}{2^{n-1}}$$

non posso non fissare m . Altrimenti c'è un attacco



Osservazioni

- Il teorema assume che tutti i messaggi abbiano la stessa taglia.
- Questa e' un'ipotesi necessaria per la validita' del teorema.
- Si puo' facilmente verificare che se si considerano messaggi di lunghezza arbitraria CBC-MAC non e' piu' sicuro.



Contro esempio

- Si consideri il seguente avversario che fa solo due domande e rompe CBC-MAC
 - $M_1 = 0^n$ (msg di un solo blocco)
 - $M_2 = M_2[1] || \dots || M_2[l]$ (msg arbitrario di l blocchi)
- Siano tag_1 e tag_2 le risposte ricevute
- A restituisce $(M_3, tag_3) = (M_2 || tag_2, tag_1)$

$$M_1 = 0^u$$

$$\text{tag}_1 \leftarrow E_K(0^u)$$

$$C[0] \leftarrow 0^n$$

for $i = 1, \dots, m$

$$C[i] \leftarrow E_K(C[i-1] \oplus M[i])$$

return $C[m]$ // Tag = $C[m]$

$$M_2 = X \in \{0,1\}^u$$

$$\text{tag}_2 \leftarrow E_K(X)$$

$$\underline{M_3} = M_2 \parallel \text{tag}_2$$

$$\underline{\text{tag}_3} \leftarrow \text{tag}_1$$

$$M_3 = X \parallel \underline{E_K(X)}$$

$$\underline{\text{tag}_3} = \underline{E_K(0^u)}$$

$$C[1] \leftarrow E_K(0^u \oplus X) = E_K(X)$$

$$C[2] \leftarrow E_K(C[1] \oplus E_K(X)) =$$

$$= E_K(E_K(X) \oplus E_K(X)) = E_K(0^u)$$

CBC-MAC version randomized

$$\text{MAC}_K(M)$$

if $M \neq M$ output $\perp$

$$M = M[1] \parallel \dots \parallel M[m]$$

$$R = C[0] \leftarrow_R \{0,1\}^u$$

for $i = 1$ to m

$$C[i] \leftarrow E_K(C[i-1] \oplus M[i]);$$

Return $(R, C[m])$

$$\text{Ver}_F(\pi, (R, C))$$

$$\pi = \pi[1] \parallel \dots \parallel \pi[m]$$

$$C[0] \leftarrow R$$

$$\text{for } i=1 \text{ to } m \quad C[i] \leftarrow E_F(C[i-1] \oplus \pi[i])$$

$$\text{if } (C[m] == C) \text{ return } 1$$

$$\text{else return } 0$$

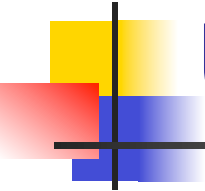
$$\pi_1 \rightarrow \underline{(R_1, C_1)} \quad \underline{C_1 = C[1] = E_F(R \oplus \pi_1)}$$

$$\pi_2, R_2 \quad \underline{\pi_2 \oplus R_2 = R_1 \oplus \pi_1} \quad \underline{R_2 = R_1 \oplus \pi_1 \oplus \pi_2}$$

$$C[0] \leftarrow R_2$$

$$\pi_2 (R_2, C_1)$$

$$C[1] = \underline{E_F(R_2 \oplus \pi_2)} = \underline{E_F(R_1 \oplus \pi_1)} = C_1$$



Ulteriori osservazioni

- A differenza di CBC\$, CBC-MAC è deterministico.
- Questa proprietà è **cruciale** per la sicurezza di CBC-MAC

IDEA: Se (R, tag) MAC di $M[1] \dots M[m]$, calcolo (R', tag) MAC di $M'[1] \dots M[m]$

$$R \oplus M[1] = R' \oplus M'[1]$$

CBC-MAC per messaggi di lunghezza variabile

Primo approccio: utilizzare chiavi (computazionalmente) indipendenti per messaggi di lunghezza diversa.

F pseudorandom function, ℓ lunghezza del messaggio m .

1. $k_\ell = F_k(\ell)$

2. $\text{tag} = \text{CBC-MAC}(k_\ell, m)$

AD OGNI LUNGHEZZA
UNA CHIAVE

CBC-MAC per messaggi di lunghezza variabile

$|M||H$

Secondo approccio: "prependere" a m la sua lunghezza (stringa di n bit).

COMINCIO A PRE COMPUTARE

n 256 bit

256 bit

Terzo approccio: due chiavi indipendenti k_1, k_2

1. $t = \text{CBC-MAC}(k_1, m)$

$\swarrow$

$C(m)$

2. $\text{tag} = \text{CBC-MAC}(k_2, t)$

un solo blocco

- Pro: non è necessario conoscere $|m|$ in anticipo
- Contro: 2 chiavi (ma si può ovviare usando prf)

CBC-MAC È TROPPO LENTO

MAC from hash functions

- Abbiamo visto come costruire MAC da cifrari a blocchi
- Possiamo ottenere MAC anche da funzioni hash costruite secondo la metodologia Markle-Damgard

Pro: La maggior parte delle funzioni hash utilizzate in pratica segue tale metodologia

Contro: Dobbiamo assumere che la funzione di compressione abbia proprietà di pseudocasualità

HASH DA COMPRESSIONE

Merkle-Damgard

$H(\cancel{K}, M)$

$y \leftarrow \text{pad}(M)$

Sia $y = M_1 || M_2 || \dots || M_n$
// $(|M_i| = b)$

$V \leftarrow \text{IV}$ $\rightarrow$ arbitrario

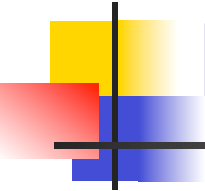
// IV vettore iniziale di v -bit

for $i = 1, \dots, n$

$V \leftarrow h(\cancel{K}, M_i || V)$

Return V

$\rightarrow K$ se c'è



HMAC - Preliminari

Facciamo le seguenti ipotesi

- H, h restituiscano output di n bit.
- Se m è nell'insieme $\{0,1\}^b$, $H(m)$ richiede una sola esecuzione di h .
- **IV, opad, ipad** costanti di lunghezza b .
 - opad contiene il byte 0x36 (ripetuto quanto serve)
 - ipad contiene il byte 0x5C (come sopra)

S di grande bob le bene

HMAC

KeyGen

- ~~$s \leftarrow \text{HASH-KEYGEN}$~~ . Δ hash-keygen
- Sceglie una stringa casuale k di b bit.
 - b lunghezza del blocco

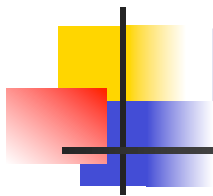
$\text{MAC}((\cancel{k}, m), m) \quad |m|=L$

m bit

$$\text{tag} \leftarrow H_s \left((k \oplus \text{opad}) \parallel \underbrace{H_s(k \oplus \text{ipad} \parallel m)}_{\text{PARTE PIÙ PESANTE}} \right)$$

b

PARTE PIÙ PESANTE



HMAC in pratica

- Molto efficiente e facile da implementare
- Standard industriale, molto utilizzato in pratica.
- Nasce come alternativa a CBC-MAC considerato troppo lento
 - Questo comportava che spesso in pratica si utilizzassero costruzioni del tutto insicure
 - Es: $\text{tag} = H_s(k || m)$ (perché non è sicuro?)

A CASA! HASH-MERKLE DAMGARD

K_1 chiave MAC

K_2 chiave di
cifatura

I Soluzione NON SICURA, MAC SEMPRE UGUALE

$tag \leftarrow Mac(K_1, m)$

$c \leftarrow Enc(K_2, m)$

Output (tag, c)

$MDec(K_1, K_2, (c, tag))$

$m \leftarrow Dec(K_2, c)$

$b \leftarrow VF(K_1, m, tag)$

if $b == 1$ output m
else output $\perp$

II Soluzione

$c \leftarrow Enc(K_2, m)$

$tag \leftarrow Mac(K_1, c)$

Output (tag, c)

$MDec(K_1, K_2, (c, tag))$

$b \leftarrow VF(K_1, c, tag)$

if $b == 0$ output $\perp$

else $m \leftarrow Dec(K_2, c)$
output m

IL CRITTOTESTO CAMBIA SEMPRE, PERCHÉ
MAC DIVERSO

$m_0, m_1 \rightarrow c, tag$

$m_0, m_1 \rightarrow \hat{c}, \hat{tag}$

FINE

II Prova